Mobile Applications

6. Maps and location-based services in Android

Boni García

boni.garcia@uc3m.es

Telematic Engineering Department
School of Engineering

2025/2026

uc3m | Universidad Carlos III de Madrid

CC BY NC SA

Table of contents

1. Introduction
2. Location-based services
3. Google Maps Platform
4. Takeaways

1. Introduction

- In this unit, we study two types of features for Android apps:

1. Location-based services
   - Accessing the device's location
   - Location providers (GPS, Wi-Fi, etc.)
   - Geocoding and reverse geocoding
2. Google Maps Platform
   - Google Maps: display maps and markers
   - Google Places: search nearby places
   - Google Directions: compute routes

Table of contents

1. Introduction
2. Location-based services
   - Emulated location
   - Locations permissions
   - Locations providers
   - Location listener
   - Geocoding
3. Google Maps Platform
4. Takeaways

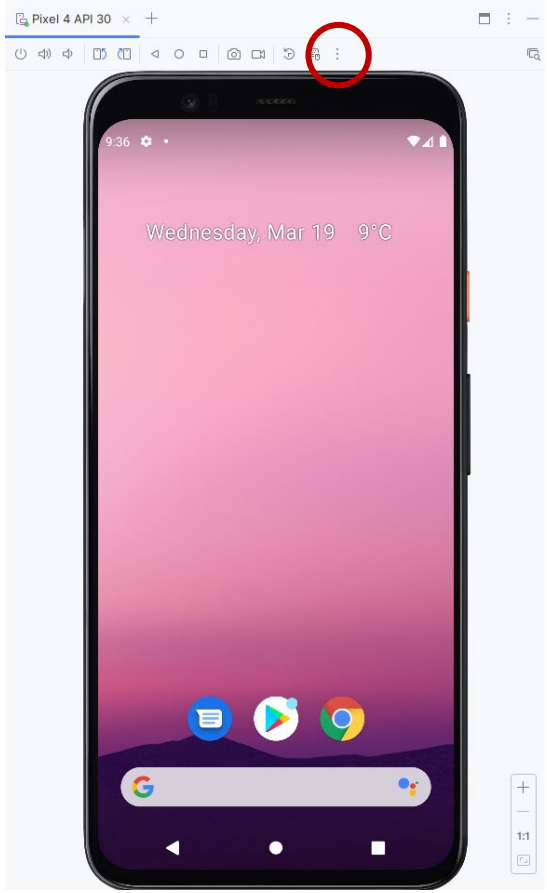
2. Location-based services

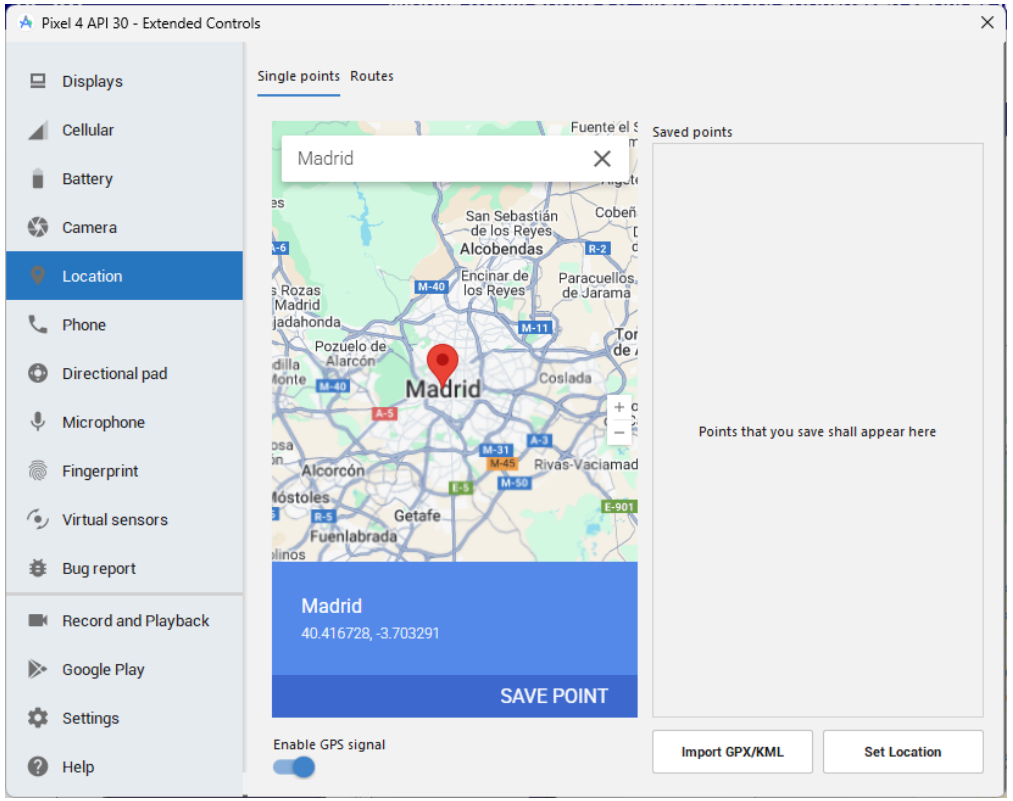
- A **location-based service** is a feature in a mobile app that uses the device's geographic location
- In this unit, we study:
  - How to emulate **device location** using an Android emulator
  - The **permissions** required to access the device location
  - The most common types of **location providers** in Android
  - How to implement a **location listener** (i.e., an app that tracks location changes)
  - How to implement **geocoding** in Android (i.e., translating an address into coordinates and vice versa)

https://developer.android.com/training/location

2. Location-based services - Emulated location

- Android Studio allows to change the location of the device, and even simulate routes





2. Location-based services - Location permissions

- To protect user privacy, apps that use location services must request location **permissions**
- There are different types of permissions regarding the location accuracy:
  - Precise: access to the device's GPS coordinates
    - Pro: It provides accurate real-time location
    - Cons: It consumes more battery

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

  - Approximate: using the network address (wifi or cellular), since Internet Service Provider (ISP) assigns IP addresses tied to a regional network node
    - Pro: It is more battery friendly
    - Cons: It provides an approximate location

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

As usual, we declare these permission in the manifest file

2. Location-based services - Location providers

- Depending on the device, Android offers different ways to determine the current location
  - These alternatives are implemented through location providers
  - In Kotlin or Java, location providers can be accessed using the `LocationManager` class
- Some common locations providers in Android are:
  - `LocationManager.`*`GPS_PROVIDER`*: based on the Global Positioning System (GPS), using coordinates obtained from satellites
  - `LocationManager.`*`NETWORK_PROVIDER`*: determines location using nearby cell towers and Wi-Fi access points
  - `LocationManager.`*`FUSED_PROVIDER`*: combines data from several location sources to provide the best possible location

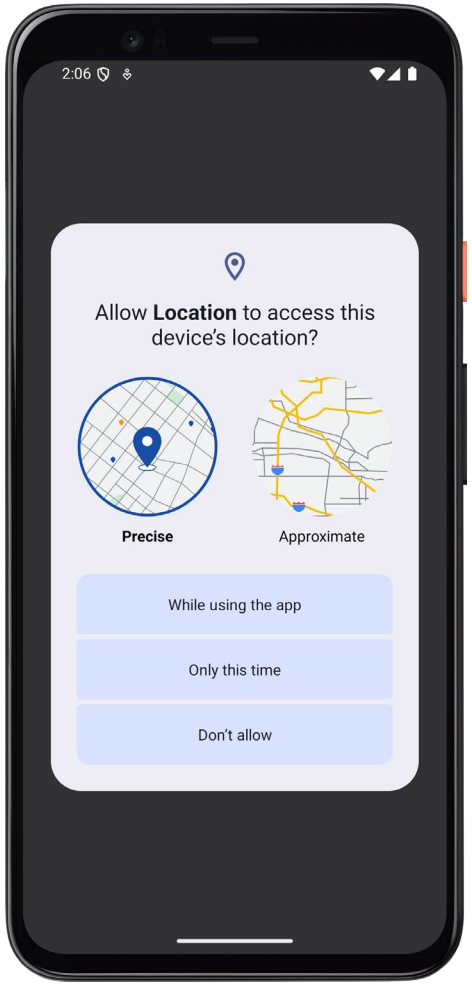
https://developer.android.com/reference/android/location/LocationManager

2. Location-based services - Location listener

Fork me on GitHub

```
@Composable
fun MainScreen(modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val permission = Manifest.permission.ACCESS_FINE_LOCATION
    val locationManager = remember {
        context.getSystemService(Context.LOCATION_SERVICE) as LocationManager
    }

    var locationText by remember { mutableStateOf("") }
    var permissionsGranted by remember {
        mutableStateOf(
            ContextCompat.checkSelfPermission(
                context, permission
            ) == PackageManager.PERMISSION_GRANTED
        )
    }
    var startListening by remember { mutableStateOf(false) }

    val permissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { isGranted ->
        permissionsGranted = isGranted
        if (isGranted) {
            startListening = true
        } else {
            Toast.makeText(
                context, context.getString(R.string.permission_denied), Toast.LENGTH_LONG
            ).show()
        }
    }
```

First, we need to request location permissions to the user

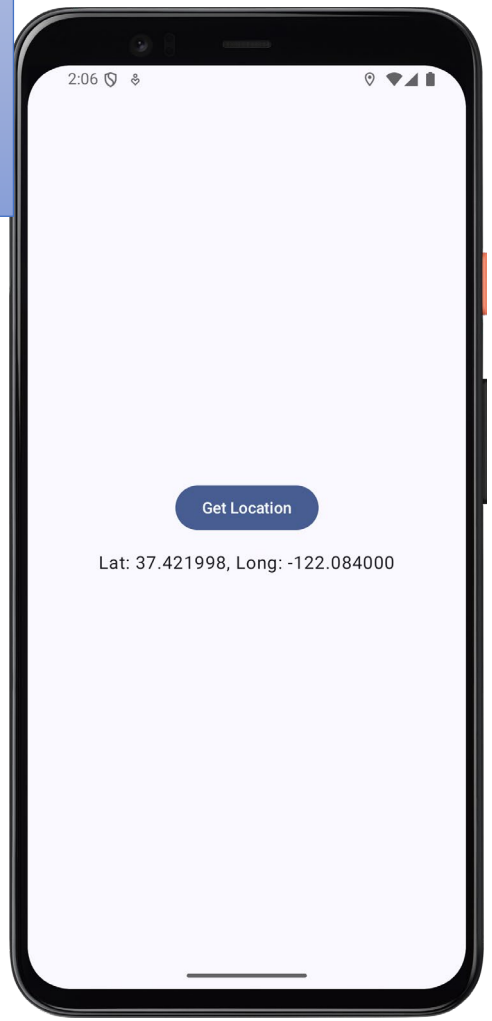


2. Location-based services - Location listener

Fork me on GitHub

Second, we use a location provider to get the device's location and request location updates

```
if (permissionsGranted && startListening) {
        DisposableEffect(Unit) {
            val provider = LocationManager.GPS_PROVIDER
            val listener = LocationListener { location: Location ->
                locationText =
                    context.getString(R.string.lat_lon, location.latitude, location.longitude)
            }
            try {
                locationManager.getLastKnownLocation(provider)?.let { location ->
                    locationText =
                        context.getString(R.string.lat_lon, location.latitude, location.longitude)
                } ?: run {
                    locationText = context.getString(R.string.waiting)
                }
                locationManager.requestLocationUpdates(
                    provider, 5000L, 10f, listener
                )
            } catch (e: SecurityException) {
                locationText = e.message ?: context.getString(R.string.permission_error)
            }
            onDispose {
                locationManager.removeUpdates(listener)
            }
        }
    }
```



`DisposableEffect(Unit)` is a composable for working with external resources that must be cleaned up. In this case, it stops receiving location updates when the composable is disposed

2. Location-based services - Geocoding

- **Geocoding** is the process of converting human-readable addresses (like "1600 Amphitheatre Parkway, Mountain View, CA") into geographic coordinates (like latitude 37.423021 and longitude -122.083739)
  - This process is sometimes referred as forward geocoding
  - Some uses of geocoding are: to place markers on a map, or centering a map
- **Reverse geocoding** is the process of converting geographic coordinates into a human-readable address
- We use the `Geocoder` class to implement both features in Android apps
  - We also need to declare the Internet connection permission

```
<uses-permission android:name="android.permission.INTERNET" />
```

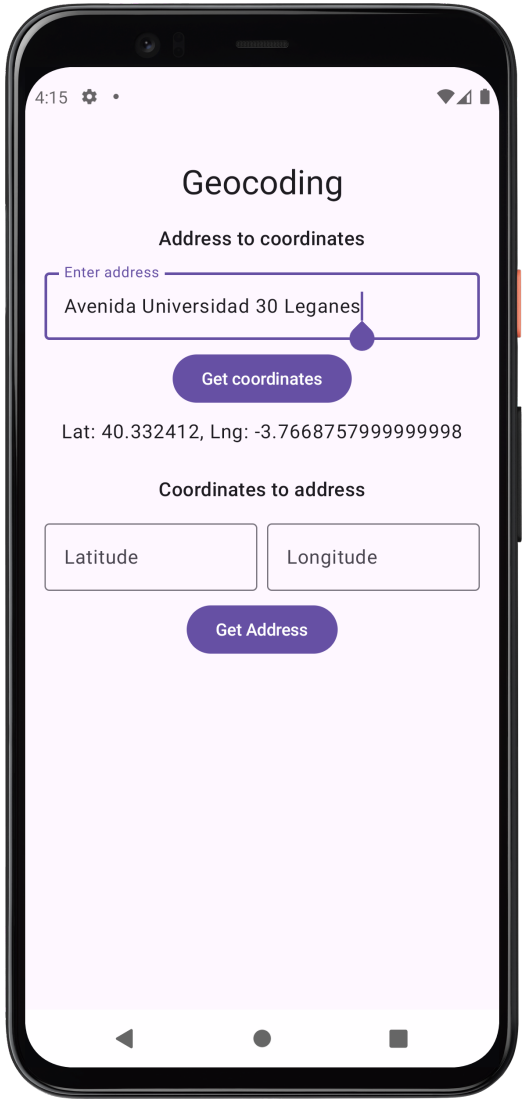
https://developers.google.com/maps/documentation/geocoding/

2. Location-based services - Geocoding

Fork me on GitHub

```
class GeocodingService(context: Context) {

    private val appContext = context.applicationContext
    private val geocoder = Geocoder(appContext, Locale.getDefault())

    suspend fun geocodeAddress(address: String): Result<String> = withContext(Dispatchers.IO) {
        try {
            val results = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
                suspendCancellableCoroutine<List<Address>?> { cont ->
                    geocoder.getFromLocationName(address, 1) { list ->
                        if (cont.isActive) cont.resume(list)
                    }
                }
            } else {
                geocoder.getFromLocationName(address, 1)
            }
            if (results.isNullOrEmpty()) {
                Result.failure(NoSuchElementException(getString(R.string.no_results)))
            } else {
                val location = results[0]
                Result.success(
                    getString(
                        R.string.lat_lon, location.latitude, location.longitude
                    )
                )
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

}
```

The geocoding logic is implemented in a separate class



Fork me on GitHub

2. Location-based services - Geocoding

```
class MyViewModel(private val geocodingService: GeocodingService) : ViewModel() {

    private val _address = MutableStateFlow("")
    val address: StateFlow<String> = _address.asStateFlow()

    private val _coordinates = MutableStateFlow("")
    val coordinates: StateFlow<String> = _coordinates.asStateFlow()

    private val _errorMessage = MutableStateFlow("")
    val errorMessage: StateFlow<String> = _errorMessage.asStateFlow()

    fun geocodeAddress(address: String) {
        viewModelScope.launch {
            val result = geocodingService.geocodeAddress(address)
            result.onSuccess { latLon ->
                _coordinates.value = latLon
                _errorMessage.value = ""
            }.onFailure { error ->
                _coordinates.value = ""
                setErrorMessage(error.message)
            }
        }
    }

    fun reverseGeocode(lat: Double, lng: Double) {
        viewModelScope.launch {
            val result = geocodingService.reverseGeocode(lat, lng)
            result.onSuccess { addressText ->
                _address.value = addressText
                _errorMessage.value = ""
            }.onFailure { error ->
                _address.value = ""
                setErrorMessage(error.message)
            }
        }
    }
}
```

Then, a `ViewModel` use the previous service

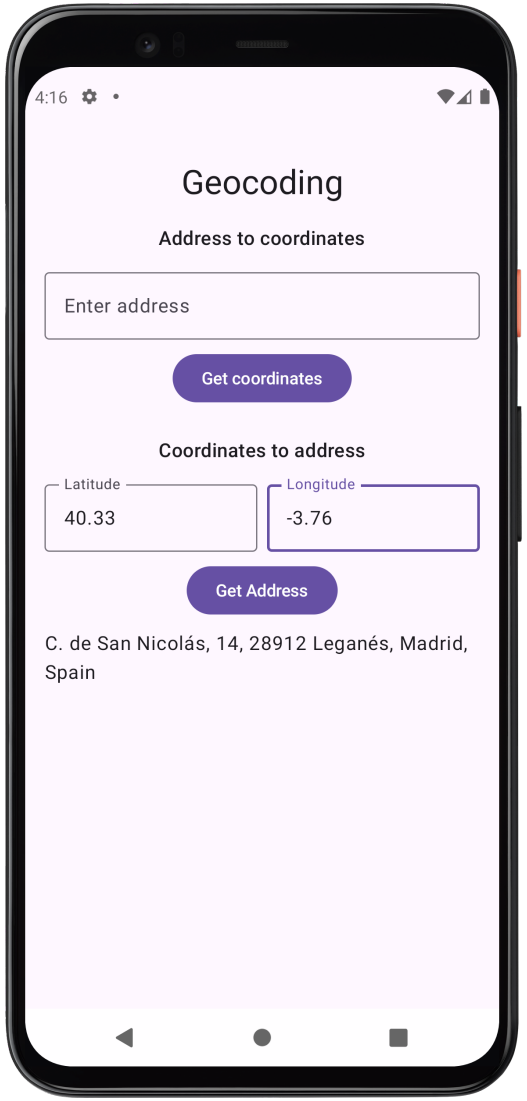


Table of contents

1. Introduction
2. Location-based services
3. Google Maps Platform
   - Google Play Services
   - Google Maps
   - Google Places
   - Google Directions
4. Takeaways

3. Google Maps Platform

- The Google Maps Platform (previously called Google Maps API) is a set of APIs and SDKs that allows to develop map-based services
- Some of the services provided by the Google Maps Platforms are:
  - Satellite imagery, aerial photography, street maps, 360° interactive panoramic views of streets (Street View), real-time traffic conditions, or route planning
- We can use the Google Maps Platforms for different types of apps, such as web, mobile, or desktop applications

Google Maps Platform

https://developers.google.com/maps

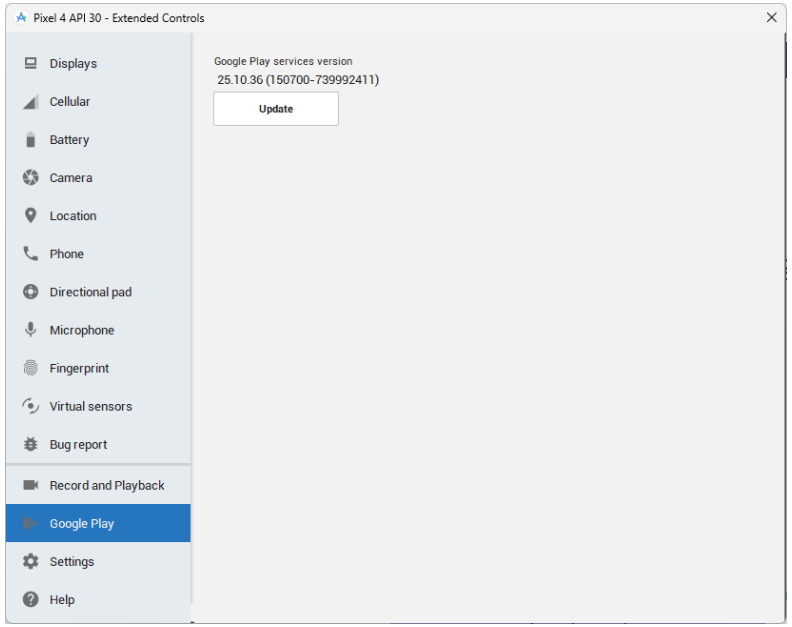
3. Google Maps Platform - Google Play Services

- The Google Maps SDK on Android depends on the Google Play services
  - Google Play Services is a background service produced by Google for Android devices
  - These services include maps and location, single sign-on account services, user health and fitness tracking, payment processing, integrated advertising or security scanning

Google Play services

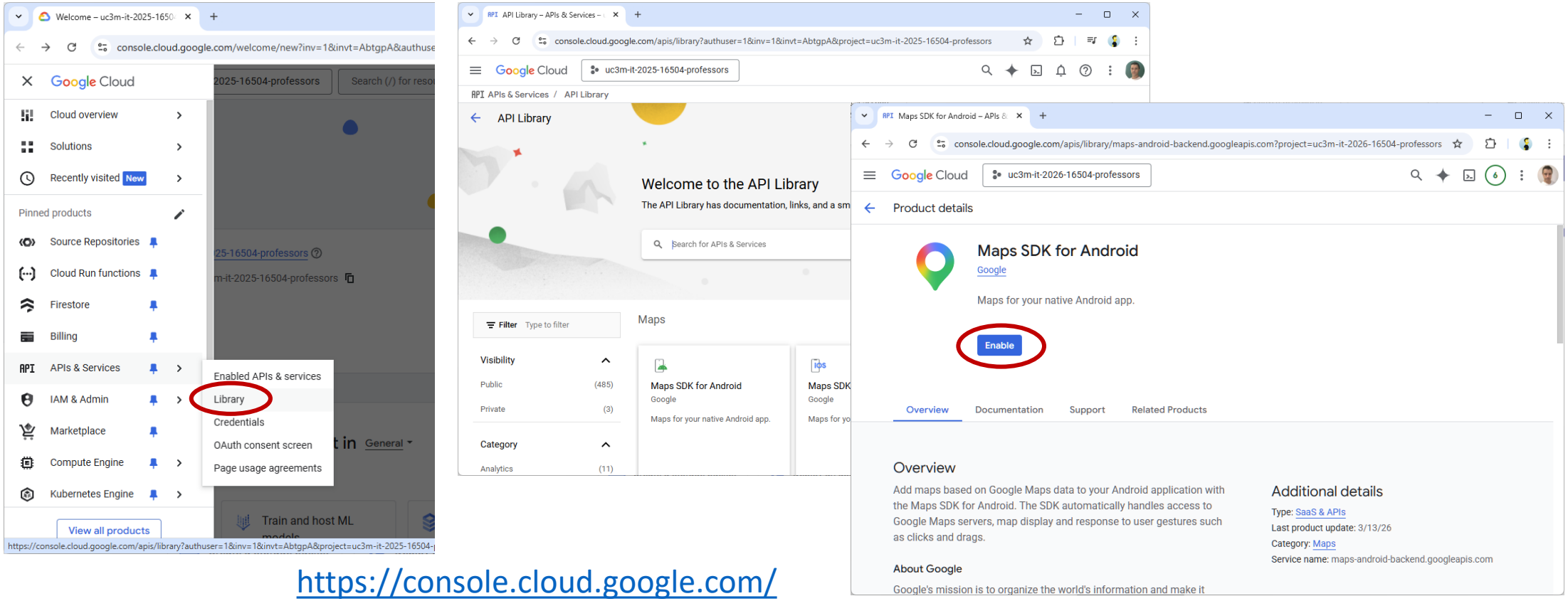
https://developers.google.com/android/

We need to ensure that Google Play is enabled in the emulator with use for development in Android Studio



3. Google Maps Platform - Google Maps

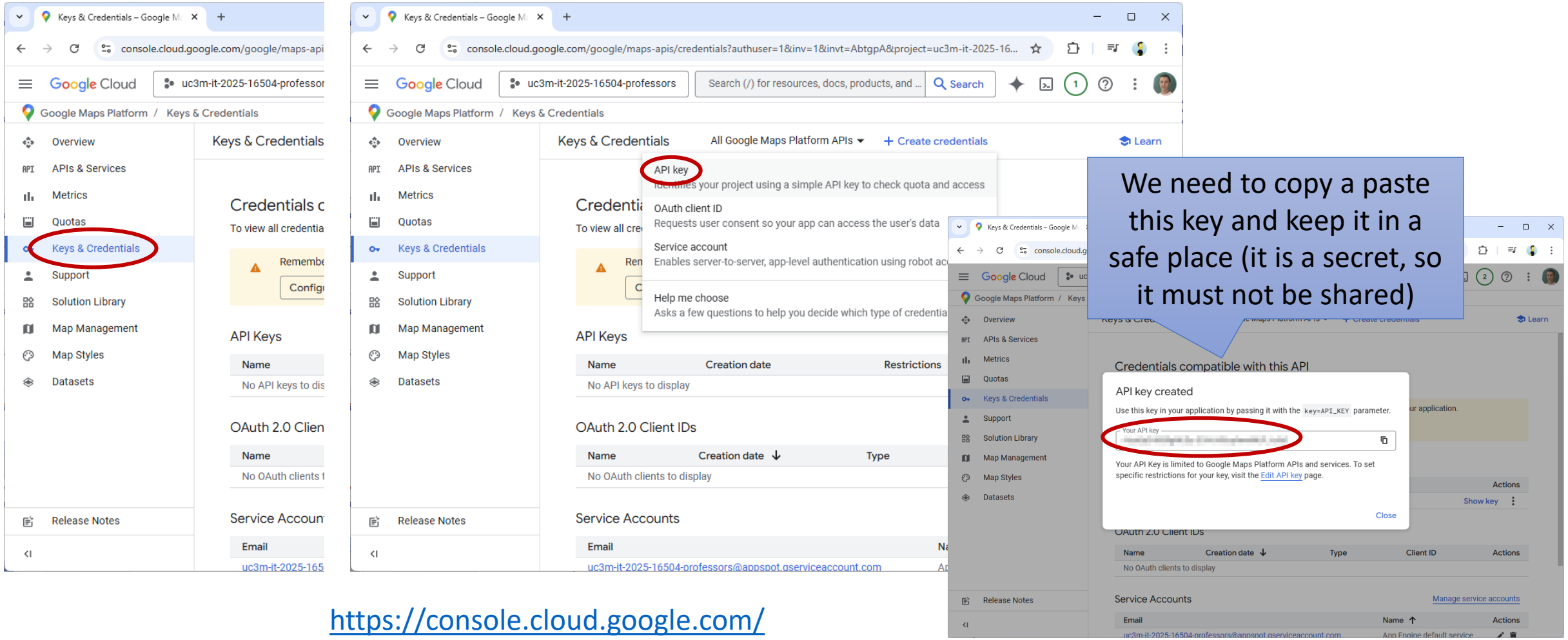
- To use any API of Google Maps Platform, first we need we need enable this SDK in the Google Cloud console
  - APIs & Services → Library



https://console.cloud.google.com/

3. Google Maps Platform - Google Maps

- Then, we need to create an API key:



https://console.cloud.google.com/

3. Google Maps Platform - Google Maps

Fork me on GitHub

- To use Google Maps in an Android app, we need to declare the following dependency in our `build.gradle.kts` (app):

build.gradle.kts (app)

```
dependencies {
    implementation(libs.maps.compose)
    implementation(libs.play.services.maps)
}
```

libs.version.toml

```
[versions]
mapsCompose = "8.2.2"
playServicesMaps = "20.0.0"

[libraries]
maps-compose = { module = "com.google.maps.android:maps-compose", version.ref = "mapsCompose" }
play-services-maps = { module = "com.google.android.gms:play-services-maps", version.ref = "playServicesMaps" }
```

- In addition, we need to declare this permissions in the manifest:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- Finally, we need to declare an API Key in our manifest:

```
<application
    <meta-data
        android:name="com.google.android.geo.API_KEY"
        android:value="${MAPS_API_KEY}" />
</application>
```

For security, it is not a good practice to hard-code the API key in the manifest

3. Google Maps Platform - Google Maps

Fork me on GitHub

- We can use a Gradle plugin to store the API key safely in the `local.properties` file (which in local, and should not be public):

build.gradle.kts (project)

```
plugins {
    // ...
    alias(libs.plugins.secrets.gradle) apply false
}
```

build.gradle.kts (app)

```
plugins {
    // ...
    alias(libs.plugins.secrets.gradle)
}

android {
    // ...
    buildFeatures {
        // ...
        buildConfig = true
    }
}
```

libs.version.toml

```
[versions]
secrets-gradle = "2.0.1"

[plugins]
secrets-gradle = { id = "com.google.android.libraries.mapsplatform.secrets-gradle-plugin", version.ref = "secrets-gradle" }
```

local.properties

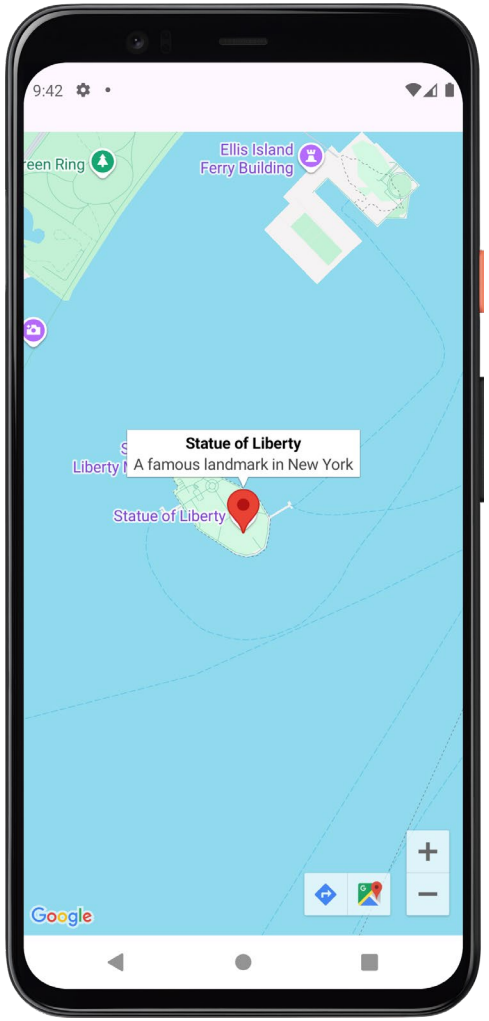
```
MAPS_API_KEY=<my-apy-key>
```

3. Google Maps Platform - Google Maps

Fork me on GitHub

```
@Composable
fun MapScreen(modifier: Modifier = Modifier) {
    val statueOfLiberty = LatLng(40.6892, -74.0445)
    val cameraPositionState = rememberCameraPositionState {
        position = CameraPosition.fromLatLngZoom(statueOfLiberty, 15f)
    }
    val markerState = rememberUpdatedMarkerState(position = statueOfLiberty)

    GoogleMap(
        modifier = modifier.fillMaxSize(), cameraPositionState = cameraPositionState
    ) {
        Marker(
            state = markerState,
            title = stringResource(R.string.title),
            snippet = stringResource(R.string.description)
        )
    }
}
```

This basic demo renders a map with Google Maps in a fixed coordinates a with a marker



https://developers.google.com/maps/documentation/android-sdk/maps-compose

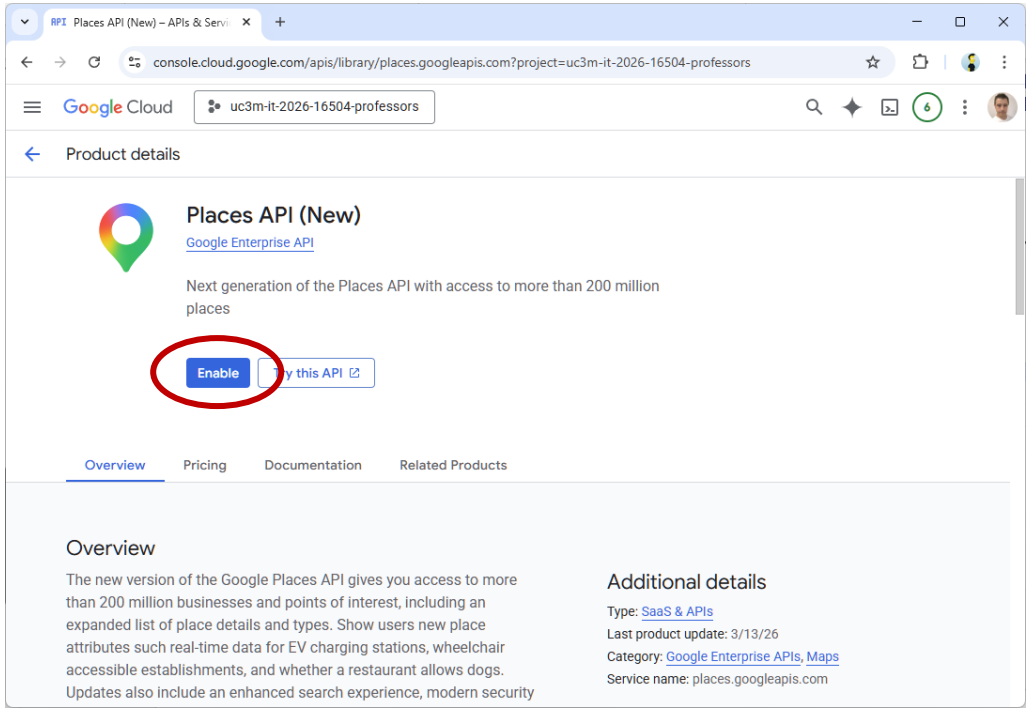
3. Google Maps Platform - Google Places

- The **Google Places API** is a web service provided by Google that allows developers to access detailed information about places (businesses, landmarks, geographic locations, etc.)
  - We are going to consume this API using a Java wrapper library (i.e., not requesting directly the web service API)
- Google Places is part of the Google Maps Platform and enables applications to search for places, retrieve place details, or auto-complete place names based on user input

https://developers.google.com/maps/documentation/places/android-sdk/overview

3. Google Maps Platform - Google Places

- To use the Google Places API (New), we need to enable it in the Google Cloud console:
  - APIs & Services → Library



https://console.cloud.google.com/

The old Places SDK is now legacy and can no longer be enabled

3. Google Maps Platform - Google Places

- To use Google Places, we need to declare the following permissions in the manifest:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- Then, we need to include the following dependency in our project:

build.gradle.kts (app)

```
dependencies {
    implementation(libs.places)
}
```

libs.version.toml

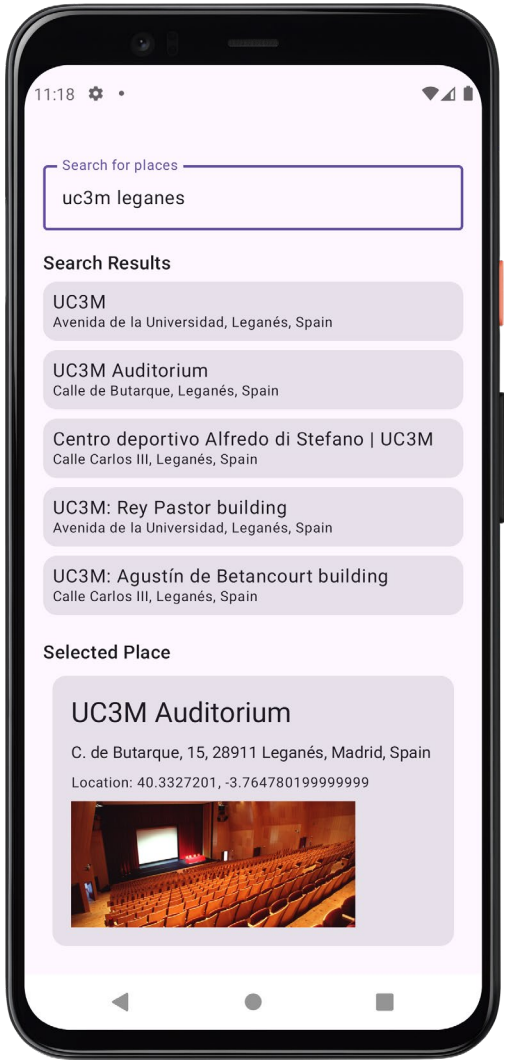
```
[versions]
places = "5.1.1"

[libraries]
places = { module = "com.google.android.libraries.places:places", version.ref = "places" }
```

3. Google Maps Platform - Google Places

Fork me on GitHub

```
class MyViewModel(application: Application) : AndroidViewModel(application) {
    private var placesClient: PlacesClient

    private val _predictions = MutableStateFlow<List<PlaceAutocomplete>>(emptyList())
    val predictions: StateFlow<List<PlaceAutocomplete>> = _predictions

    private val _selectedPlace = MutableStateFlow<PlaceDetails?>(null)
    val selectedPlace: StateFlow<PlaceDetails?> = _selectedPlace

    init {
        if (!Places.isInitialized()) {
            Places.initializeWithNewPlacesApiEnabled(
                application, BuildConfig.MAPS_API_KEY
            )
        }
        placesClient = Places.createClient(application)
    }

    fun searchPlaces(query: String) {
        viewModelScope.launch {
            try {
                val request = FindAutocompletePredictionsRequest.builder().setQuery(query).build()
                val response = placesClient.findAutocompletePredictions(request).await()
                _predictions.value = response.autocompletePredictions.map { prediction ->
                    PlaceAutocomplete(
                        placeId = prediction.placeId,
                        primaryText = prediction.getPrimaryText(null).toString(),
                        secondaryText = prediction.getSecondaryText(null).toString()
                    )
                }
            } catch (e: Exception) {
                e.printStackTrace()
            }
        }
    }
    //...
}
```

We also need an API key for Google Places (we can use the same previously created)



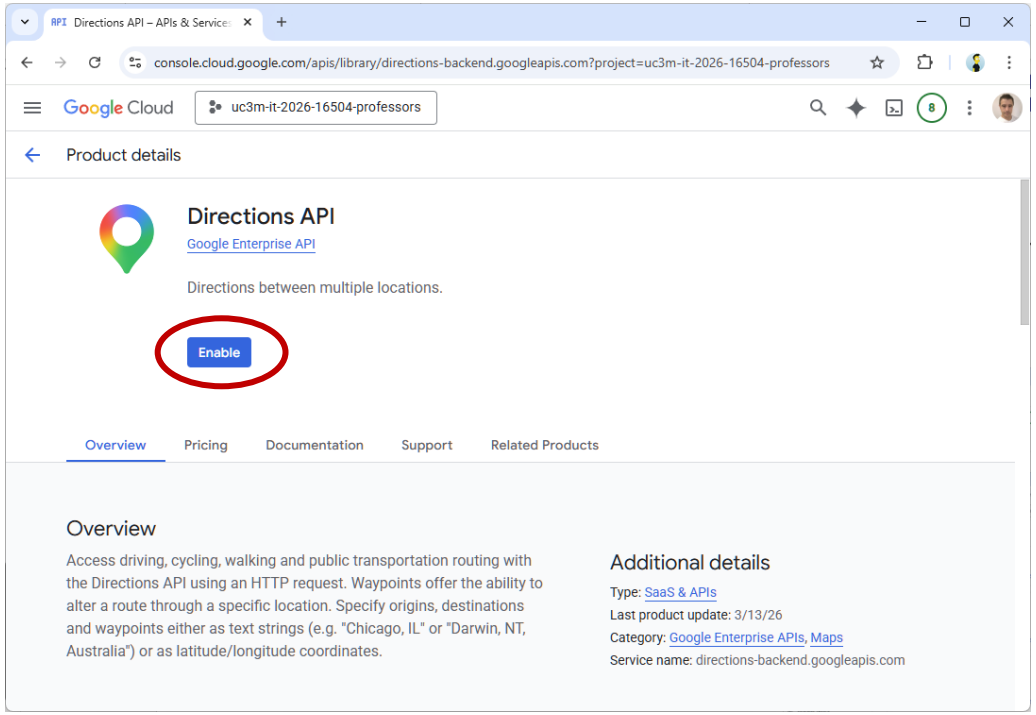
3. Google Maps Platform - Google Directions

- The **Google Directions API** is a web service provided by Google as part of the Google Maps Platform that calculates directions between multiple locations
  - It returns detailed route information, including travel time, distance, turn-by-turn navigation steps, and even alternative routes for different travel modes (driving, walking, cycling, or public transit)
  - Like we do with Google Places, we consume this API using a Java wrapper library (i.e., not requesting directly the web service API)

https://developers.google.com/maps/documentation/directions

3. Google Maps Platform - Google Directions

- To use Google Directions, first we need to enable it in the Google Cloud console:
  - APIs & Services → Library



https://console.cloud.google.com/

3. Google Maps Platform - Google Directions

- To use Google Directions, we need to declare the following permissions in the manifest:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- Then, we need the following dependency:

build.gradle.kts (app)

```
dependencies {
    implementation(libs.google.google.maps.services)
}
```

libs.version.toml

```
[versions]
googleGoogleMapsServices = "2.2.0"

[libraries]
google-google-maps-services = { module = "com.google.maps:google-maps-services", version.ref = "googleGoogleMapsServices" }
```

3. Google Maps Platform - Google Directions

Fork me on GitHub

```
fun getDirections(
    origin: String, destination: String, callback: (DirectionsResult?, String?) ->
Unit
) {
    val context = GeoApiContext.Builder().apiKey(BuildConfig.MAPS_API_KEY).build()

    DirectionsApi.newRequest(context).mode(TravelMode.DRIVING).origin(origin)
        .destination(destination).alternatives(true)
        .setCallback(object : PendingResult.Callback<DirectionsResult> {
            override fun onResult(result: DirectionsResult) {
                callback(result, null)
            }

            override fun onFailure(e: Throwable) {
                callback(null, e.message)
            }
        })
}
```

```
@Composable
fun DrawRoute(route: DirectionsRoute) {
    // Convert the polyline points to LatLng objects
    val pathPoints = remember(route) {
        route.overviewPolyline.decodePath().map { LatLng(it.lat, it.lng) }
    }

    // Draw the polyline
    Polyline(
        points = pathPoints,
        color = Color.Blue,
        width = 8f
    )
}
```

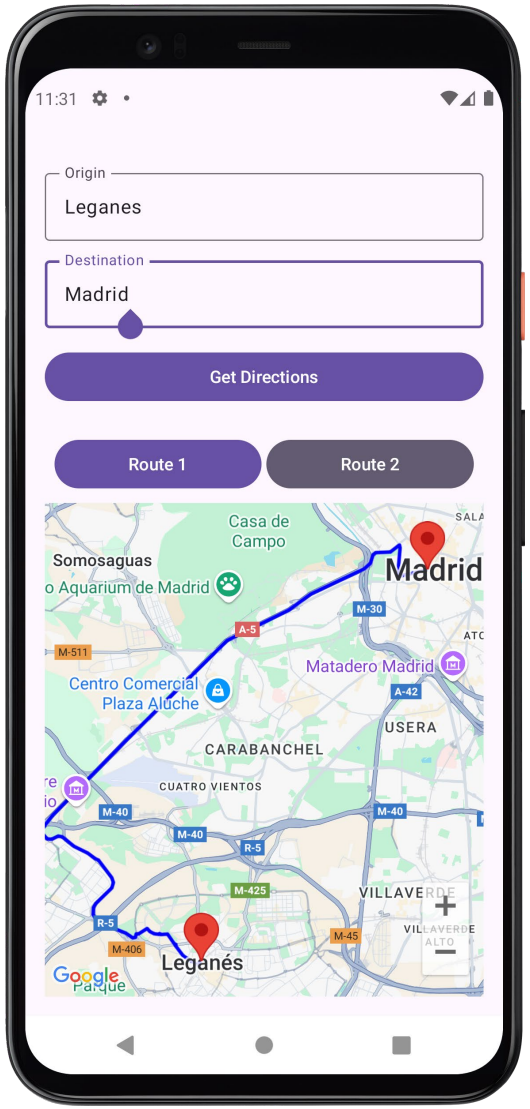


Table of contents

1. Introduction
2. Location-based services
3. Google Maps Platform
4. Takeaways

4. Takeaways

- Location-based services allows mobile device applications to known where the device is geographically located
- There are different location providers in Android, such as GPS or network based
- We can use location-based services to implement different features like location listener or geocoding (forward or reverse) in Android
- The Google Maps Platform is a set of APIs and SDKs that allows to develop map-based services
- We can use these APIs in Android app to implement map-based features, such as render maps and put markers on it, search places, or get directions